

A Formal Algorithmic Analysis of Queue Data Structures: Implementations, Deques, and Sliding Window Maximum Optimizations

Abstract

Queues represent a fundamental linear data structure operating under the First-In, First-Out (FIFO) protocol. This paper presents a formal analysis and reference implementations of queue structures using dynamic arrays and singly-linked lists, detailing their performance bounds. We investigate circular buffer representations using modular arithmetic to achieve constant-time operations. We also formalize the implementation of a queue using two stacks, establishing their amortized complexity bounds. Finally, we explore the Sliding Window Maximum problem, demonstrating how the Monotonic Deque paradigm reduces the naive $O(N \cdot K)$ brute-force solution to an optimal $O(N)$ time complexity. Complete implementations in Java and formal correctness arguments are provided.

Index Terms

linked list, deep copy, random pointer, hash map, node interleaving, Java, algorithm analysis, computational complexity



1 Introduction

Linear data structures form the foundation of computer science, providing structured mechanisms for memory management and element access. While the Stack data structure operates under the Last-In, First-Out (LIFO) access pattern, the Queue data structure enforces a First-In, First-Out (FIFO) order. In a FIFO structure, the element that has been in the queue for the longest duration is the first to be removed.

Queues are utilized in a variety of systems programming and network design scenarios:

- Operating System Scheduling: Ready queues manage processes awaiting CPU execution (e.g., Round-Robin scheduling).
- Network Packet Buffering: Routers maintain packets in transmission queues to regulate bandwidth allocation.
- Asynchronous Task Management: Message brokers (e.g., RabbitMQ, Kafka) utilize message queues to decouple distributed services.
- Algorithm Design: Breadth-First Search (BFS) uses a queue to traverse graphs layer by layer.

This paper provides a rigorous algorithmic treatment of queues. Section 2 details the Queue Abstract Data Type (ADT) and its mathematical states. Section 3 and Section 4 analyze array-based (circular) and linked-list-based queue implementations. Section 5 formalizes a queue implementation using two stacks with an amortized complexity proof. Section 6 discusses the Doubly Ended Queue (Deque). Finally, Section 7 details the Sliding Window Maximum problem, presenting an optimal $O(N)$ solver using a Monotonic Deque.

2 Queue Abstract Data Type (ADT)

A queue Q is a sequence of elements supporting restricted boundaries. Let $Q = \langle q_1, q_2, \dots, q_n \rangle$ represent the queue, where q_1 is at the front (head) and q_n is at the rear (tail).

The Queue ADT supports the following primary operators:

- Enqueue(Q, x): Inserts element x at the rear. $\langle q_1, \dots, q_n \rangle \xrightarrow{\text{enqueue}(x)} \langle q_1, \dots, q_n, x \rangle$.
- Dequeue(Q): Removes and returns the front element q_1 . $\langle q_1, \dots, q_n \rangle \xrightarrow{\text{dequeue}()} \langle q_2, \dots, q_n \rangle$.
- Peek(Q): Returns the front element q_1 without altering the queue.
- IsEmpty(Q): Returns true if $n = 0$, else false.

Table 1 summarizes the asymptotic time and space bounds for different queue designs.

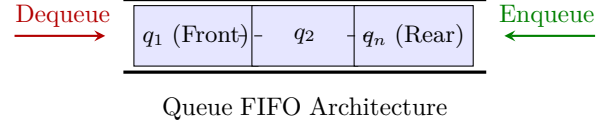


Fig. 1. Visual representation of a Queue operating under the First-In, First-Out (FIFO) principle.

TABLE 1
Asymptotic Complexity of Queue Implementations.

Operation	Array-based (Shifting)	Circular Array	Linked List-based
Enqueue	$O(1)$ amortized	$O(1)$ worst-case	$O(1)$ worst-case
Dequeue	$O(N)$ worst-case	$O(1)$ worst-case	$O(1)$ worst-case
Peek	$O(1)$ worst-case	$O(1)$ worst-case	$O(1)$ worst-case
Space Complexity	$O(N)$ total	$O(N)$ total	$O(N)$ total
Auxiliary Space	$O(1)$	$O(1)$	$O(1)$

3 Array-Based Queue Implementation

Implementing a queue using a static array of size C requires careful indexing. If dequeue always returns the element at index 0, then all remaining elements must shift left by one spot, incurring an $O(N)$ cost.

To achieve $O(1)$ time complexity for both enqueue and dequeue operations, we utilize a Circular Queue model. We maintain a front cursor f and a rear cursor r , initialized such that the queue is empty. By treating the array as a closed loop and using modular arithmetic, pointers advance circularly:

$$\text{rear} \leftarrow (\text{rear} + 1) \bmod C \quad (1)$$

$$\text{front} \leftarrow (\text{front} + 1) \bmod C \quad (2)$$

3.1 Java Implementation

The Java implementation of a circular array queue is as follows:

```
public class CircularQueue {
    private final int[] arr;
    private int front;
    private int rear;
    private int size;
    private final int capacity;

    public CircularQueue(int capacity) {
        this.capacity = capacity;
        this.arr = new int[capacity];
        this.front = 0;
        this.rear = -1;
        this.size = 0;
    }

    public boolean enqueue(int x) {
        if (isFull()) {
            return false; // Queue Overflow
        }
        rear = (rear + 1) % capacity;
        arr[rear] = x;
        size++;
        return true;
    }

    public int dequeue() {
        if (isEmpty()) {
            return -1; // Queue Underflow
        }
        int val = arr[front];
```

```

        front = (front + 1) % capacity;
        size--;
        return val;
    }

    public int peek() {
        if (isEmpty()) {
            return -1;
        }
        return arr[front];
    }

    public boolean isEmpty() {
        return size == 0;
    }

    public boolean isFull() {
        return size == capacity;
    }
}

```

4 Singly-Linked List Queue Implementation

A linked list queue avoids the fixed-capacity limitations of arrays. We maintain reference pointers to the start and end of the list:

- head pointer: Points to the front node. Dequeue operations occur here.
- tail pointer: Points to the rear node. Enqueue operations occur here.

4.1 Java Implementation

The Java implementation of a linked list queue is as follows:

```

public class LinkedListQueue {
    private static class Node {
        int data;
        Node next;
        Node(int data) {
            this.data = data;
            this.next = null;
        }
    }

    private Node head;
    private Node tail;

    public LinkedListQueue() {
        this.head = null;
        this.tail = null;
    }

    public void enqueue(int x) {
        Node newNode = new Node(x);
        if (tail == null) {
            head = newNode;
            tail = newNode;
            return;
        }
        tail.next = newNode;
        tail = newNode;
    }

    public int dequeue() {

```

```

    if (head == null) {
        return -1; // Underflow
    }
    int data = head.data;
    head = head.next;
    if (head == null) {
        tail = null; // List became empty
    }
    return data;
}

public int peek() {
    if (head == null) {
        return -1;
    }
    return head.data;
}

public boolean isEmpty() {
    return head == null;
}
}

```

5 Queue Realization Using Stacks

A common algorithmic problem is implementing a queue using two stacks, st1 and st2. This design achieves a FIFO interface using LIFO containers.

5.1 Algorithm Description

We designate st1 as the primary enqueue target and st2 as the dequeue source:

- Enqueue(x): Push x onto st1.
- Dequeue():
 - 1) If both st1 and st2 are empty, throw an underflow exception.
 - 2) If st2 is empty, transfer all elements from st1 to st2 by popping from st1 and pushing to st2. This reverses the elements' order, transforming LIFO order to FIFO.
 - 3) Pop and return the top element of st2.

5.2 Java Implementation

The Java implementation of the stack-based queue is as follows:

```
import java.util.Stack;
```

```

public class StackQueue {
    private final Stack<Integer> st1 = new Stack<>();
    private final Stack<Integer> st2 = new Stack<>();

    public void enqueue(int x) {
        st1.push(x);
    }

    public int dequeue() {
        if (isEmpty()) {
            return -1; // Underflow
        }
        if (st2.isEmpty()) {
            while (!st1.isEmpty()) {
                st2.push(st1.pop());
            }
        }
        return st2.pop();
    }
}

```

```

}

public int peek() {
    if (isEmpty()) {
        return -1;
    }
    if (st2.isEmpty()) {
        while (!st1.isEmpty()) {
            st2.push(st1.pop());
        }
    }
    return st2.peek();
}

public boolean isEmpty() {
    return st1.isEmpty() && st2.isEmpty();
}
}

```

5.3 Complexity Analysis and Amortized Proof

Theorem 1. In a two-stack queue implementation, the amortized time complexity of the Dequeue operation is $O(1)$.

Proof. Let N be the number of elements processed by the queue. We analyze the total operations performed on the underlying stacks:

- 1) Each element is pushed onto st1 exactly once during Enqueue.
- 2) Each element is popped from st1 exactly once during the transfer step.
- 3) Each element is pushed onto st2 exactly once during the transfer step.
- 4) Each element is popped from st2 exactly once during Dequeue.

Across the complete lifecycle of N elements, the total stack push and pop operations are bounded by $4N$. Since each individual stack push or pop takes $O(1)$ time, the cumulative time for all operations is $O(N)$. Distributing this cost over N dequeue operations yields an amortized time complexity of $O(1)$ per operation. $\square$

6 Doubly Ended Queue (Deque)

A Doubly Ended Queue (Deque) is a generalization of a queue that supports insertion and deletion at both the front and the rear.

The primary operations of a Deque are:

- insertFirst(x): Inserts element x at the front.
- insertLast(x): Inserts element x at the rear (equivalent to Enqueue).
- removeFirst(): Removes and returns the front element (equivalent to Dequeue).
- removeLast(): Removes and returns the rear element.

A Deque can be implemented in $O(1)$ time for all operations using either a Circular Array with two active cursors or a Doubly Linked List, where each node has pointers to its predecessor and successor.

7 Sliding Window Maximum and Monotonic Deques

The Sliding Window Maximum problem is a classic application of Deques.

7.1 Problem Statement

Given an integer array A of size N and a sliding window of size K ($1 \leq K \leq N$), we find the maximum value in each contiguous subarray of size K . There are exactly $N - K + 1$ such windows.

7.2 Brute-Force vs. Optimized Monotonic Solver

A brute-force solution checks every window and searches for its maximum element:

```
// Brute Force: O(N * K)
public int[] bruteForce(int[] A, int K) {
    int[] res = new int[A.length - K + 1];
    for (int i = 0; i <= A.length - K; i++) {
        int max = A[i];
        for (int j = i; j < i + K; j++) {
            max = Math.max(max, A[j]);
        }
        res[i] = max;
    }
    return res;
}
```

In the worst case (e.g., $K \approx N/2$), the time complexity is quadratic: $O(N \cdot K) \approx O(N^2)$.

To achieve an optimal linear time complexity, we maintain a Deque of array indices. The indices are stored such that their corresponding array values are in strictly decreasing order. When processing element $A[i]$:

- 1) We pop indices from the front of the Deque if they are out of the current window (i.e., index $< i - K + 1$).
- 2) We pop indices from the rear of the Deque if their values are less than or equal to $A[i]$ (i.e., $A[\text{idx}] \leq A[i]$), as these elements are redundant for the current and any future window.
- 3) We append the current index i to the rear of the Deque.
- 4) The maximum value of the active window is always located at the front of the Deque.

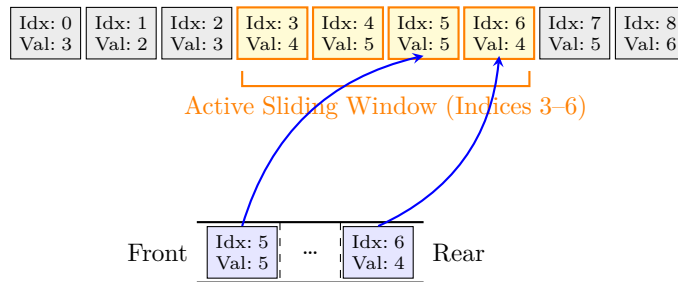
7.3 Illustrative Execution Trace

We trace the evaluation of $A = [3, 2, 3, 4, 5, 5, 4, 5, 6]$ with window size $K = 4$ in Table 2.

TABLE 2
Dry-run trace of Sliding Window Maximum algorithm for $K = 4$.

Step i	$A[i]$	Deque State (indices / values)	Output Max	Action Comments
0	3	$\langle 0/(3) \rangle$	–	Push index 0
1	2	$\langle 0/(3), 1/(2) \rangle$	–	Push index 1
2	3	$\langle 2/(3) \rangle$	–	Pop index 1 (val $2 \leq 3$), pop index 0 (val $3 \leq 3$)
3	4	$\langle 3/(4) \rangle$	$A[3] = 4$	Pop index 2 (val $3 \leq 4$), push index 3
4	5	$\langle 4/(5) \rangle$	$A[4] = 5$	Pop index 3 (val $4 \leq 5$), push index 4
5	5	$\langle 5/(5) \rangle$	$A[5] = 5$	Pop index 4 (val $5 \leq 5$), push index 5
6	4	$\langle 5/(5), 6/(4) \rangle$	$A[5] = 5$	Push index 6 (val $4 < 5$)
7	5	$\langle 7/(5) \rangle$	$A[7] = 5$	Pop index 6, pop index 5, push index 7
8	6	$\langle 8/(6) \rangle$	$A[8] = 6$	Pop index 7, push index 8

Input Array A showing Window at $i = 6$ ($K = 4$)



Monotonic Deque (Decreasing order)

Fig. 2. Monotonic Deque status at step $i = 6$ for the sliding window of size $K = 4$. The indices in the deque are stored in decreasing order of their corresponding values: $A[5] = 5 > A[6] = 4$. The maximum value for this window is located at the front of the deque (Index 5, Value 5).

7.4 Java Reference Implementation

The Java implementation of the optimal sliding window maximum algorithm is as follows:

```
import java.util.Deque;
import java.util.ArrayDeque;

public class SlidingWindowMaximum {
    public static int[] maxSlidingWindow(int[] A, int K) {
        if (A == null || A.length == 0 || K <= 0) {
            return new int[0];
        }
        int N = A.length;
        int[] res = new int[N - K + 1];
        int idx = 0;
        Deque<Integer> dq = new ArrayDeque<>();

        for (int i = 0; i < N; i++) {
            // Remove elements that are out of bounds of the current window
            if (!dq.isEmpty() && dq.peekFirst() < i - K + 1) {
                dq.pollFirst();
            }

            // Remove elements from the rear that are smaller than the current element A[i]
            while (!dq.isEmpty() && A[dq.peekLast()] <= A[i]) {
                dq.pollLast();
            }

            // Add current index to the rear
            dq.offerLast(i);

            // Record maximum element for current window
            if (i >= K - 1) {
                res[idx++] = A[dq.peekFirst()];
            }
        }
        return res;
    }
}
```

7.5 Complexity Analysis and Proof of Optimality

Theorem 2. The monotonic Deque sliding window solver runs in $O(N)$ time.

Proof. Let N be the size of the array A . We evaluate the amortized cost by tracking insertions and deletions on the Deque object:

- 1) Each index $i \in [0, N - 1]$ is pushed into the Deque at most once (via `offerLast`).
- 2) Each index i can be removed from the Deque at most once, either from the front (via `pollFirst` when it falls out of window boundaries) or from the rear (via `pollLast` when pruned by a larger incoming value).

Because each item is added at most once and removed at most once, the total number of operations on the Deque across the entire execution of the loop is bounded by $2N$. Since both `offer` and `poll` operations on a deque run in constant $O(1)$ time, the algorithm's runtime is bounded by $O(N)$. $\square$

The auxiliary space complexity is $O(K)$ to store indices in the Deque, which is bounded by $O(N)$ in the worst case where $K = N$.

8 Conclusion

This paper presented an algorithmic study of the Queue data structure and its core implementation architectures. We demonstrated that naive array implementations incur an $O(N)$ shifting cost, whereas circular buffer indexing and linked lists achieve optimal $O(1)$ bounds. We analyzed the realization of a queue using two stacks, proving an amortized constant-time dequeue complexity. Finally, we demonstrated the power of the Monotonic Deque paradigm, using pruning to resolve the Sliding Window Maximum problem in linear $O(N)$ time instead of quadratic $O(N \cdot K)$ brute-force complexity.